

FIAP GRADUAÇÃO

TECNOLOGIA EM DESENVOLVIMENTO DE SISTEMAS

DevOps Tools & Cloud Computing

Front-end - Spring | CI CD Script | Banco Oracle

PROF. João Menk

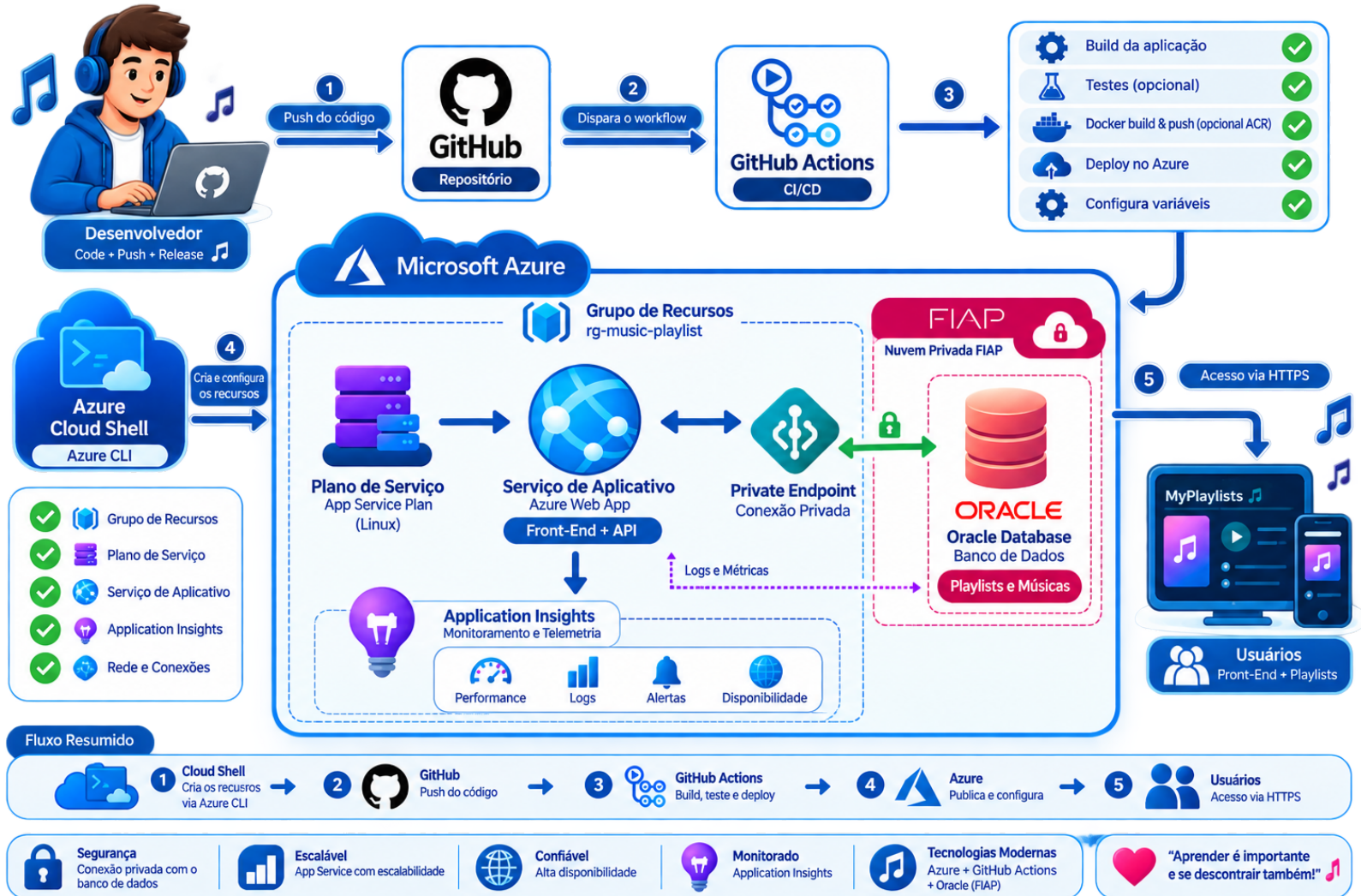
profjoao.menk@fiap.com.br

Nesse projeto, iremos Utilizar o **Cloud Shell** para implantar um App com Front-End de Playlists de Músicas, afinal, “aprender é importante e se descontrair também” (Steves Jobs) 😊

O procedimento será efetuado utilizando o **Azure CLI** para criar os recursos necessários, realizar o Deploy, configurar as Variáveis de Ambiente e criar o CI/CD com GitHub Actions. Sem esquecer de criar uma monitoração do App e Banco



Arquitetura Macro



Antes de começar vamos registrar os serviços que iremos utilizar e instalar as extensões do Azure CLI que precisamos

```
az provider register --namespace Microsoft.Web
```

```
az provider register --namespace Microsoft.Insights
```

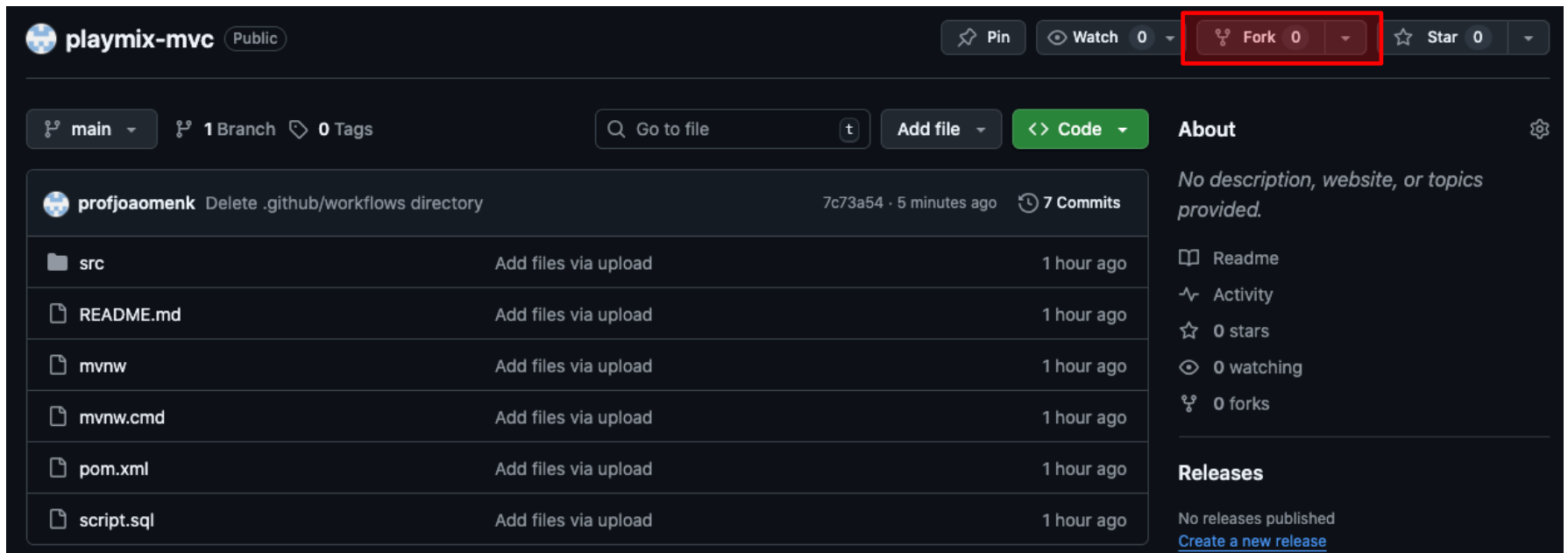
```
az provider register --namespace Microsoft.Operationallnsights
```

```
az provider register --namespace Microsoft.ServiceLinker
```

```
az extension add --name application-insights
```

Agora realize o Fork do Projeto abaixo em seu GitHub

<https://github.com/profjoaomenk/playmix-mvc.git>



The screenshot shows the GitHub repository page for 'playmix-mvc' by 'profjoaomenk'. The repository is public and has 0 stars and 0 forks. The 'Fork' button is highlighted with a red box. The repository contains a 'main' branch with 1 branch and 0 tags. The file list shows a directory 'src' and files 'README.md', 'mvnw', 'mvnw.cmd', 'pom.xml', and 'script.sql'. The 'About' section indicates no description, website, or topics are provided. The 'Releases' section shows no releases published.

File	Action	Time
src	Add files via upload	1 hour ago
README.md	Add files via upload	1 hour ago
mvnw	Add files via upload	1 hour ago
mvnw.cmd	Add files via upload	1 hour ago
pom.xml	Add files via upload	1 hour ago
script.sql	Add files via upload	1 hour ago

Vamos começar acessando a **Cloud Shell** no Portal e criando nosso Grupo de Recursos

```
# Variáveis
RESOURCE_GROUP_NAME="rg-playmix-mvc"
WEBAPP_NAME="playmix-mvc-rm9999"
APP_SERVICE_PLAN="playmix-mvc"
LOCATION="brazilsouth"
RUNTIME="JAVA:17-java17"
GITHUB_REPO_NAME="profjoaomenk/playmix-mvc"
BRANCH="main"
APP_INSIGHTS_NAME="ai-playmix-mvc"

# Criação do Grupo de Recursos
az group create \
  --name $RESOURCE_GROUP_NAME \
  --location "$LOCATION"
```

Altere para seu RM

Altere para sua região

Altere para seu Repositório

Agora vamos criar o **Application Insights** e nosso **Plano de Serviço**

Criar Application Insights

```
az monitor app-insights component create \  
  --app $APP_INSIGHTS_NAME \  
  --location "$LOCATION" \  
  --resource-group $RESOURCE_GROUP_NAME \  
  --application-type web
```

Criar o Plano de Serviço

```
az appservice plan create \  
  --name $APP_SERVICE_PLAN \  
  --resource-group $RESOURCE_GROUP_NAME \  
  --location "$LOCATION" \  
  --sku F1 \  
  --is-linux
```

Chegou a hora de criar nosso Serviço de Aplicativo e Habilitar o SCM (transferência do pacote gerado para o /home/site/wwwroot pelo GitHub Actions)

```
# Criar o Serviço de Aplicativo
az webapp create \
  --name $WEBAPP_NAME \
  --resource-group $RESOURCE_GROUP_NAME \
  --plan $APP_SERVICE_PLAN \
  --runtime "$RUNTIME"
```

az webapp list-runtimes Lista dos possíveis Runtimes (Linguagem de programação e Servidor Web)

```
# Habilita a autenticação Básica (SCM)
az resource update \
  --resource-group $RESOURCE_GROUP_NAME \
  --namespace Microsoft.Web \
  --resource-type basicPublishingCredentialsPolicies \
  --name scm \
  --parent sites/$WEBAPP_NAME \
  --set properties.allow=true
```



Agora vamos **Conectar** nosso Web App com o Application Insights

Recuperar a String de Conexão do Application Insights (assim podemos nos conectar a esse App Insights)

```
CONNECTION_STRING=$(az monitor app-insights component show \
--app $APP_INSIGHTS_NAME \
--resource-group $RESOURCE_GROUP_NAME \
--query connectionString \
--output tsv)
```

Configurar as Variáveis de Ambiente necessárias do nosso App e do Application Insights

```
az webapp config appsettings set \
--name "$WEBAPP_NAME" \
--resource-group "$RESOURCE_GROUP_NAME" \
--settings \
APPLICATIONINSIGHTS_CONNECTION_STRING="$CONNECTION_STRING" \
ApplicationInsightsAgent_EXTENSION_VERSION="~3" \
XDT_MicrosoftApplicationInsights_Mode="Recommended" \
XDT_MicrosoftApplicationInsights_PreemptSdk="1" \
SPRING_DATASOURCE_USERNAME="rm9999" \
SPRING_DATASOURCE_PASSWORD="xxxxxxx" \
SPRING_DATASOURCE_URL="jdbc:oracle:thin:@//oracle.fiap.com.br:1521/ORCL"
```



```
1 # Configura a URL de conexao com o banco de dados Oracle
2 spring.datasource.url=${SPRING_DATASOURCE_URL}
3
4 # Configura o nome de usuario para autenticao no banco de dados
5 spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
6
7 # Configura a senha para autenticao no banco de dados
8 spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
```

Reiniciar o Web App

```
az webapp restart --name $WEBAPP_NAME --resource-group $RESOURCE_GROUP_NAME
```

Criar a conexão do nosso Web App com o Application Insights

```
az monitor app-insights component connect-webapp \
--app $APP_INSIGHTS_NAME \
--web-app $WEBAPP_NAME \
--resource-group $RESOURCE_GROUP_NAME
```

Antes de começar o Build e Deploy automatizados

O Build do App será executado em um ambiente separado da Azure (GitHub), isso pode afetar a leitura das **Variáveis de Ambiente** que estão definidas no **application.properties**



Por exemplo, em nosso App a opção de criar os objetos no Banco de forma automática está ligada, então, na execução dos testes ele irá ler as Variáveis: URL, USUÁRIO e SENHA, porém elas estão preparadas para recuperar os valores de Variáveis de Ambiente, que por sua vez estão definidas somente em nosso Web App

```
1  # Configura a URL de conexao com o banco de dados Oracle
2  spring.datasource.url=${SPRING_DATASOURCE_URL}
3
4  # Configura o nome de usuario para autenticao no banco de dados
5  spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
6
7  # Configura a senha para autenticao no banco de dados
8  spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
```

Nesse cenário temos alguns caminhos a escolher:

01) As Variáveis de Ambiente definidas no **application.properties** devem ser informadas no ambiente do próprio GitHub. A partir do Deploy serão lidas pelo Web App (em: Variáveis de Ambiente)

Para essa escolha precisamos criar os Secrets no GitHub e alterar o arquivo yml criado automaticamente pela Azure (precisamos rodar uma vez o Build. Irá falhar...)

02) Como as Variáveis de Ambiente só serão lidas quando executar os testes, podemos alterar o arquivo yml gerado e realizar o Build pulando os testes, incluindo o parâmetro: *-DskipTests*

Para essa escolha precisamos também criar os objetos de Banco de forma Manual antes de executar o App

Critério	Cenário 1: Secrets GitHub	Cenário 2: -DskipTests
Integridade dos testes	Mantida (todos executados)	Perdida (testes ignorados)
Automação	Total (build e deploy totalmente CI/CD)	Parcial (executa manualmente DB)
Segurança das credenciais	Alta (GitHub Secrets criptografados)	Alta (variáveis em seu Web App)
Tempo de implementação	Moderado	Baixo
Risco de falhas em produção	Baixo	Alto

Em nosso exemplo iremos executar o caminho do GitHub Secrets e deixar documentado a escolha do Skip Tests

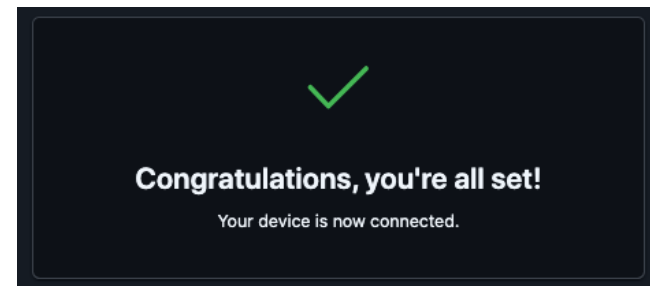
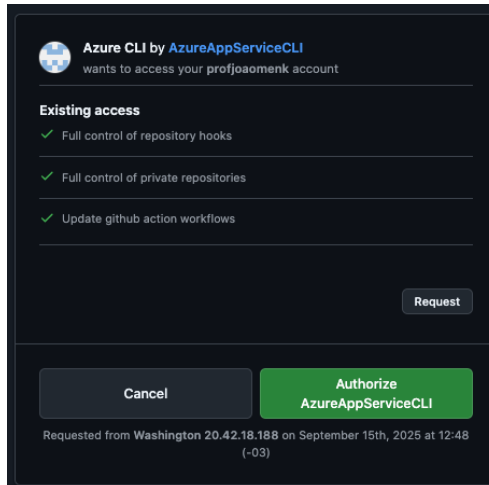
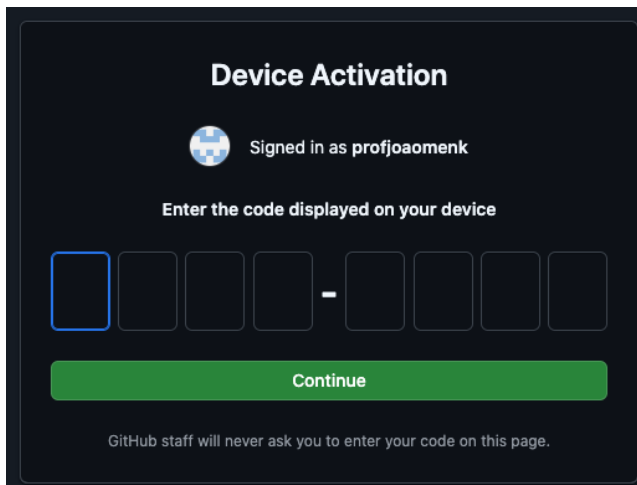
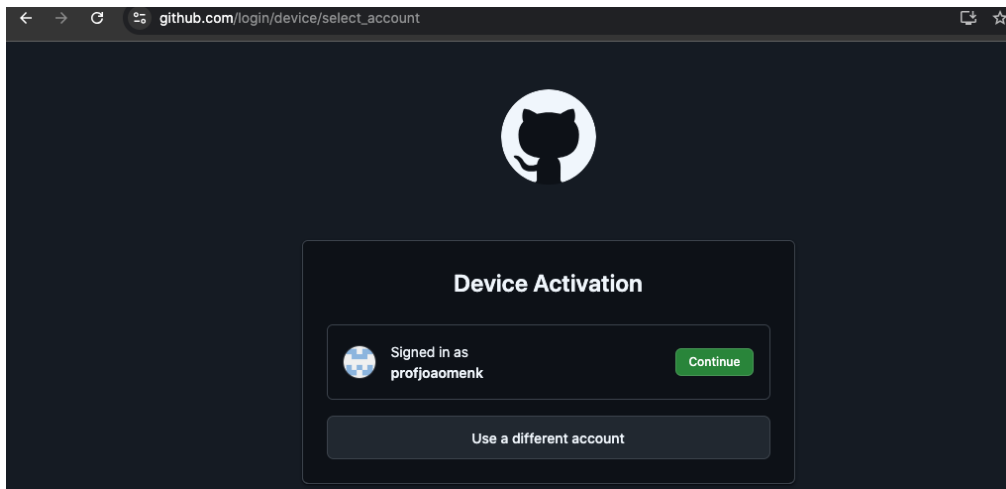
Configurar o GitHub Actions com nosso Web App, para que o Build e o Deploy seja efetuado de forma automática

Configurar GitHub Actions para Build e Deploy automático

```
az webapp deployment github-actions add \  
  --name $WEBAPP_NAME \  
  --resource-group $RESOURCE_GROUP_NAME \  
  --repo $GITHUB_REPO_NAME \  
  --branch $BRANCH \  
  --login-with-github
```

```
joao [ ~ ]$ az webapp deployment github-actions add \  
  --name $WEBAPP_NAME \  
  --resource-group $RESOURCE_GROUP_NAME \  
  --repo $GITHUB_REPO_NAME \  
  --branch $BRANCH \  
  --login-with-github  
Please navigate to https://github.com/login/device and enter the user code 9332-1237 to activate  
and retrieve your github personal access token  
Waiting up to '14' minutes for activation
```

Conceda a permissão para que o Web App acesse seu Repositório



O Build irá falhar

Seguindo a **primeira opção**, depois do primeiro Build (que falha), **no seu repositório do GitHub**:

01) Vá em **Settings** → **Secrets and variables** → **Actions**

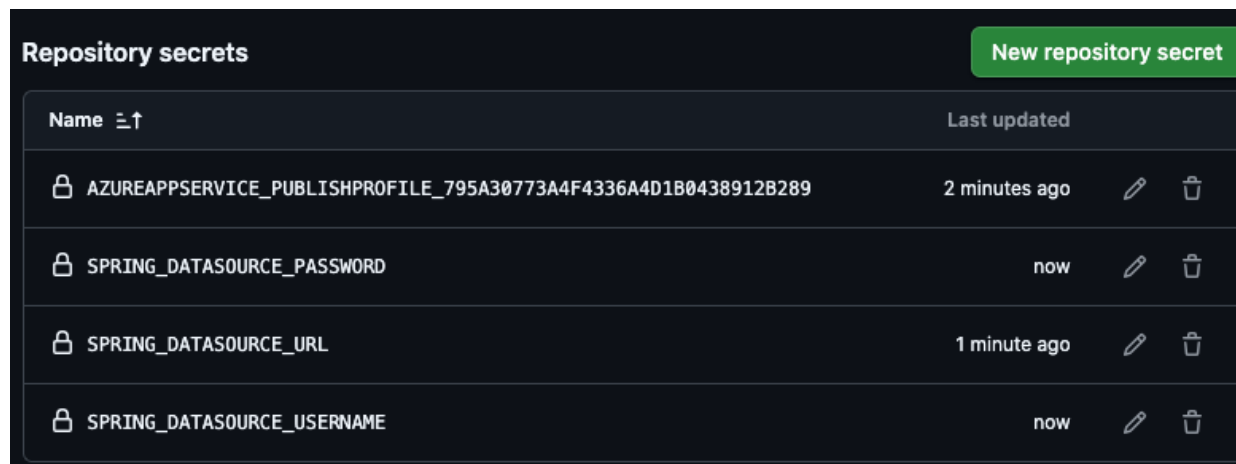
02) Clique em **New repository secret**

03) Adicione os 3 secrets (um de cada vez):

SPRING_DATASOURCE_URL=sua-url-do-banco

SPRING_DATASOURCE_USERNAME=seu-usuario

SPRING_DATASOURCE_PASSWORD=sua-senha



Agora edite seu **arquivo yml gerado** e inclua a sessão de Variáveis de Ambiente

- name: Build with Maven
run: mvn clean install

env:

- SPRING_DATASOURCE_URL: \${ secrets.SPRING_DATASOURCE_URL }
- SPRING_DATASOURCE_USERNAME: \${ secrets.SPRING_DATASOURCE_USERNAME }
- SPRING_DATASOURCE_PASSWORD: \${ secrets.SPRING_DATASOURCE_PASSWORD }

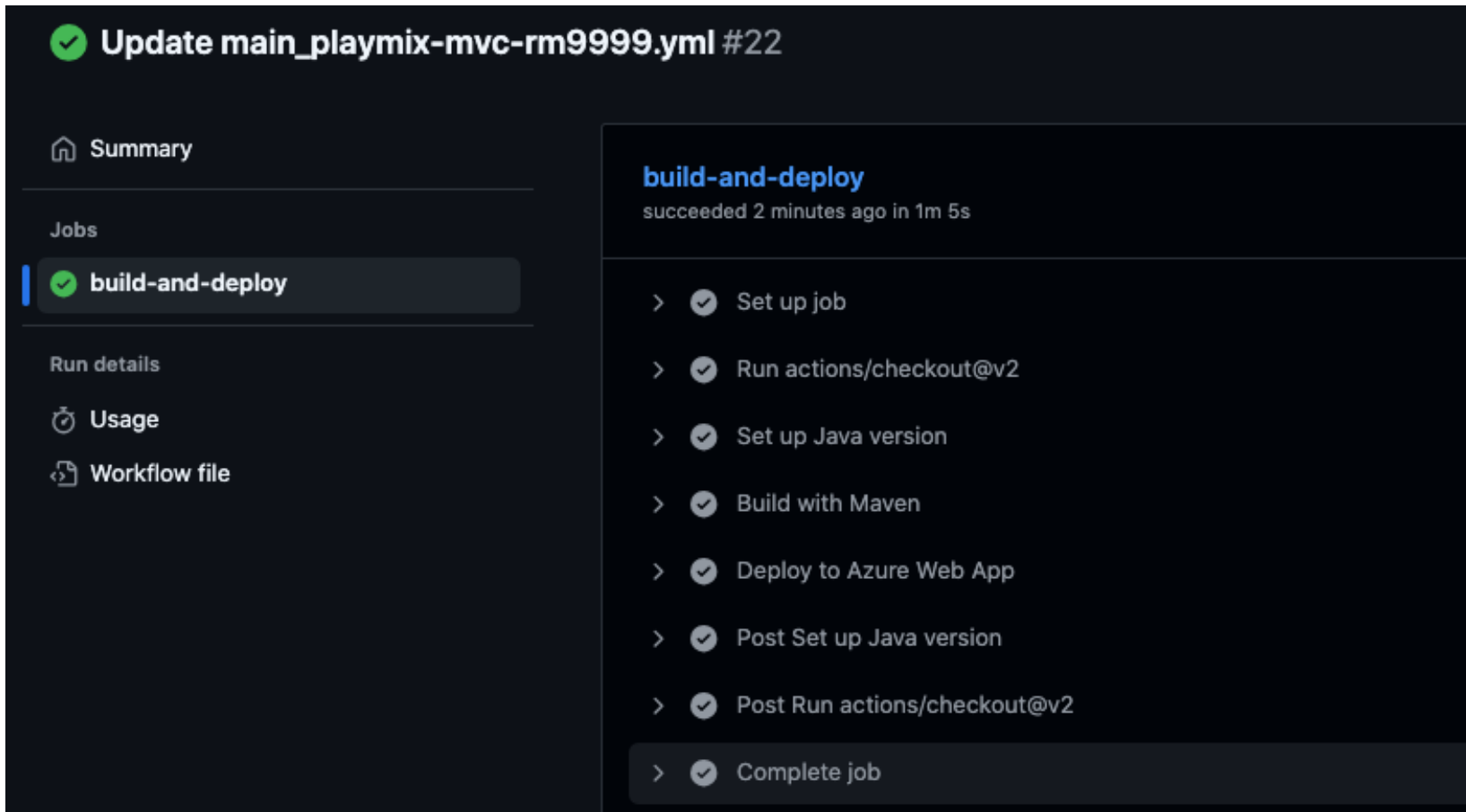
CUIDADO NA INDENTAÇÃO

```
- name: Set up Java version
  uses: actions/setup-java@v1
  with:
    java-version: '17'

- name: Build with Maven
  run: mvn clean install
  env:
    SPRING_DATASOURCE_URL: ${ secrets.SPRING_DATASOURCE_URL }
    SPRING_DATASOURCE_USERNAME: ${ secrets.SPRING_DATASOURCE_USERNAME }
    SPRING_DATASOURCE_PASSWORD: ${ secrets.SPRING_DATASOURCE_PASSWORD }

- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v2
```

Depois de realizar o commit, aguarde o Build e o Deploy de forma automática novamente



Update main_playmix-mvc-rm9999.yml #22

Summary

Jobs

- build-and-deploy**

Run details

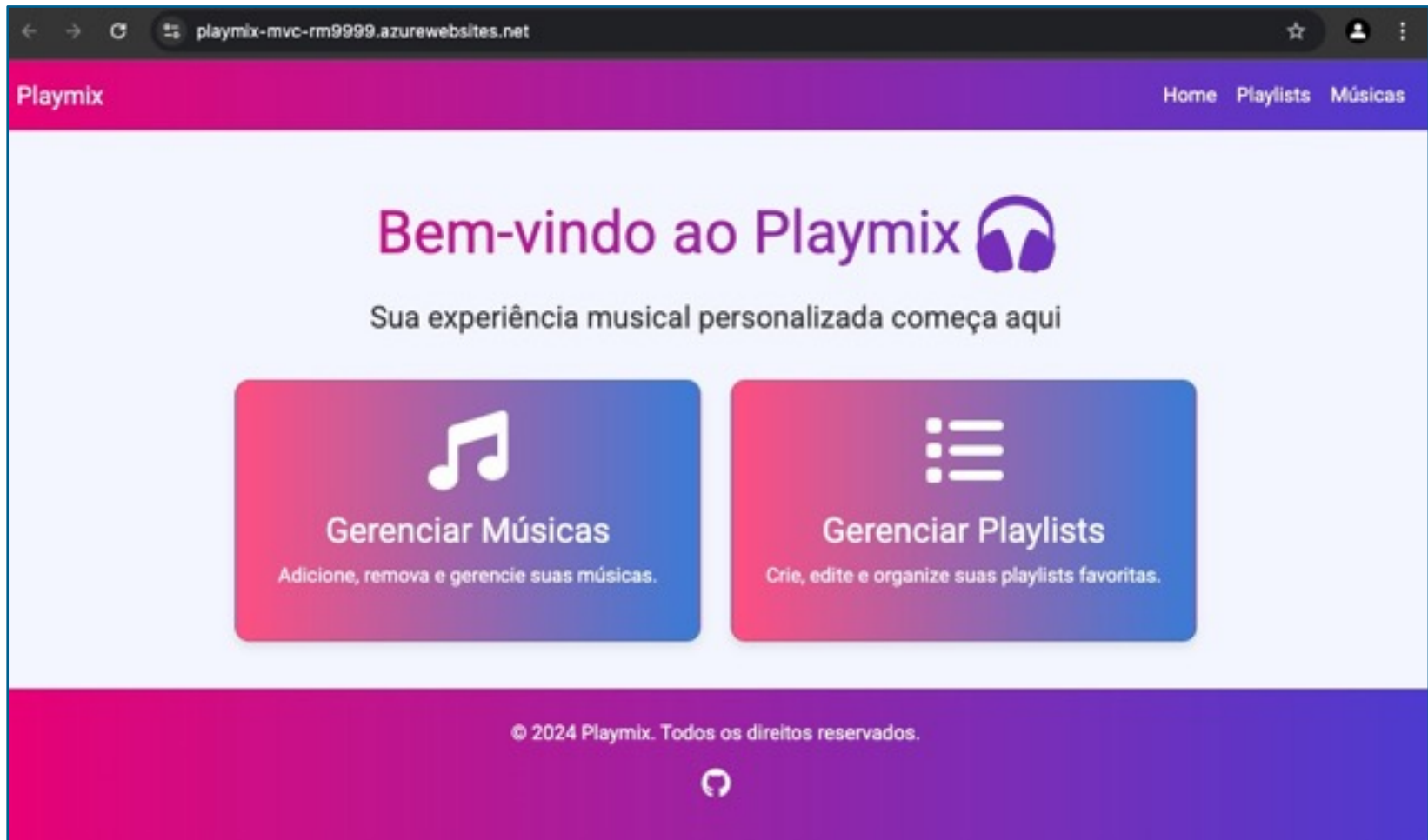
Usage

Workflow file

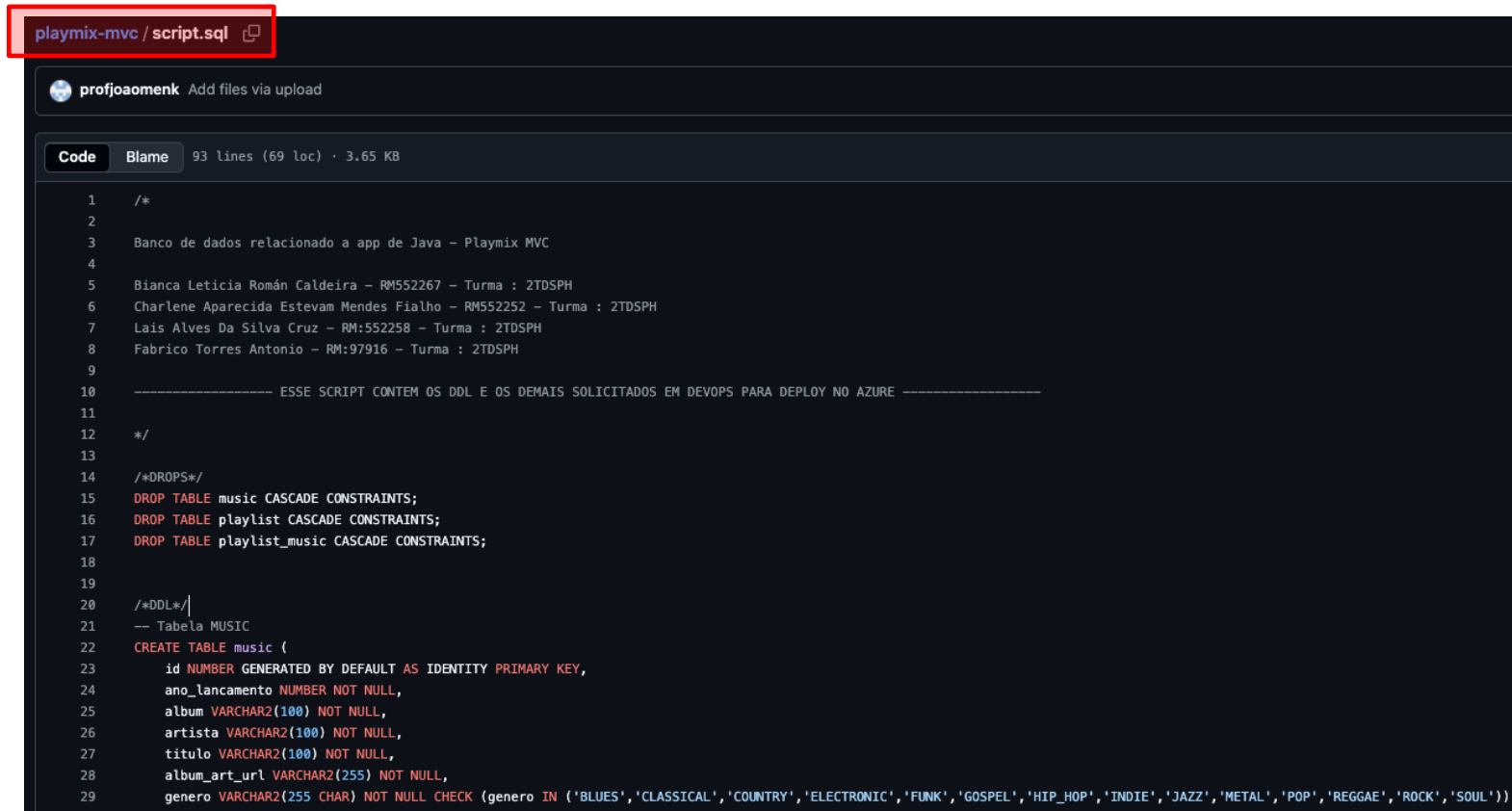
build-and-deploy
succeeded 2 minutes ago in 1m 5s

- > Set up job
- > Run actions/checkout@v2
- > Set up Java version
- > Build with Maven
- > Deploy to Azure Web App
- > Post Set up Java version
- > Post Run actions/checkout@v2
- > Complete job

Acesse o Link da Aplicação e realize os Testes



Utilizando a **segunda opção**, primeiro crie os Objetos do Banco Oracle em sua conta. Existe um arquivo no Repositório para rodar (**script.sql**). Copie, cole e execute no SQL Developer



```
playmix-mvc / script.sql

profjoaomenk Add files via upload

Code Blame 93 lines (69 loc) · 3.65 KB

1  /*
2
3  Banco de dados relacionado a app de Java - Playmix MVC
4
5  Bianca Letícia Román Caldeira - RM552267 - Turma : 2TDSPH
6  Charlene Aparecida Estevam Mendes Fialho - RM552252 - Turma : 2TDSPH
7  Lais Alves Da Silva Cruz - RM:552258 - Turma : 2TDSPH
8  Fabricio Torres Antonio - RM:97916 - Turma : 2TDSPH
9
10 ----- ESSE SCRIPT CONTEM OS DDL E OS DEMAIS SOLICITADOS EM DEVOPS PARA DEPLOY NO AZURE -----
11
12 */
13
14 /*DROPS*/
15 DROP TABLE music CASCADE CONSTRAINTS;
16 DROP TABLE playlist CASCADE CONSTRAINTS;
17 DROP TABLE playlist_music CASCADE CONSTRAINTS;
18
19
20 /*DDL*/
21 -- Tabela MUSIC
22 CREATE TABLE music (
23     id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
24     ano_lancamento NUMBER NOT NULL,
25     album VARCHAR2(100) NOT NULL,
26     artista VARCHAR2(100) NOT NULL,
27     titulo VARCHAR2(100) NOT NULL,
28     album_art_url VARCHAR2(255) NOT NULL,
29     genero VARCHAR2(255 CHAR) NOT NULL CHECK (genero IN ('BLUES','CLASSICAL','COUNTRY','ELECTRONIC','FUNK','GOSPEL','HIP_HOP','INDIE','JAZZ','METAL','POP','REGGAE','ROCK','SOUL'))
30     -- Comentários Tabela de música
```

Agora edite seu **arquivo yml gerado** e inclua a opção para pular os testes

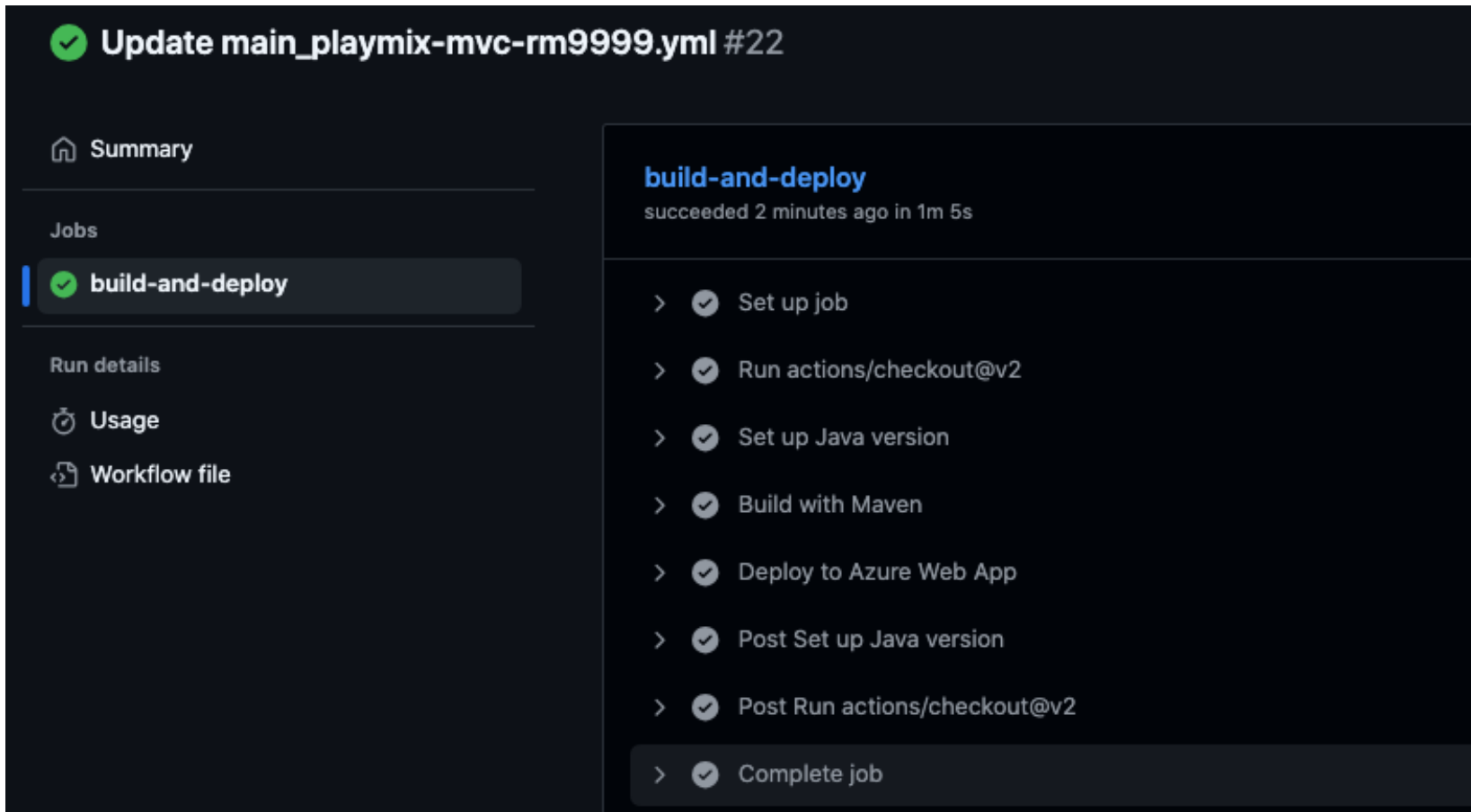
- name: Build with Maven
 - run: mvn clean install **-DskipTests**

```
- name: Set up Java version
  uses: actions/setup-java@v1
  with:
    java-version: '17'

- name: Build with Maven
  run: mvn clean install -DskipTests

- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v2
  with:
```

Depois de realizar o commit, aguarde o Build e o Deploy de forma automática novamente



Update main_playmix-mvc-rm9999.yml #22

Summary

Jobs

- build-and-deploy**

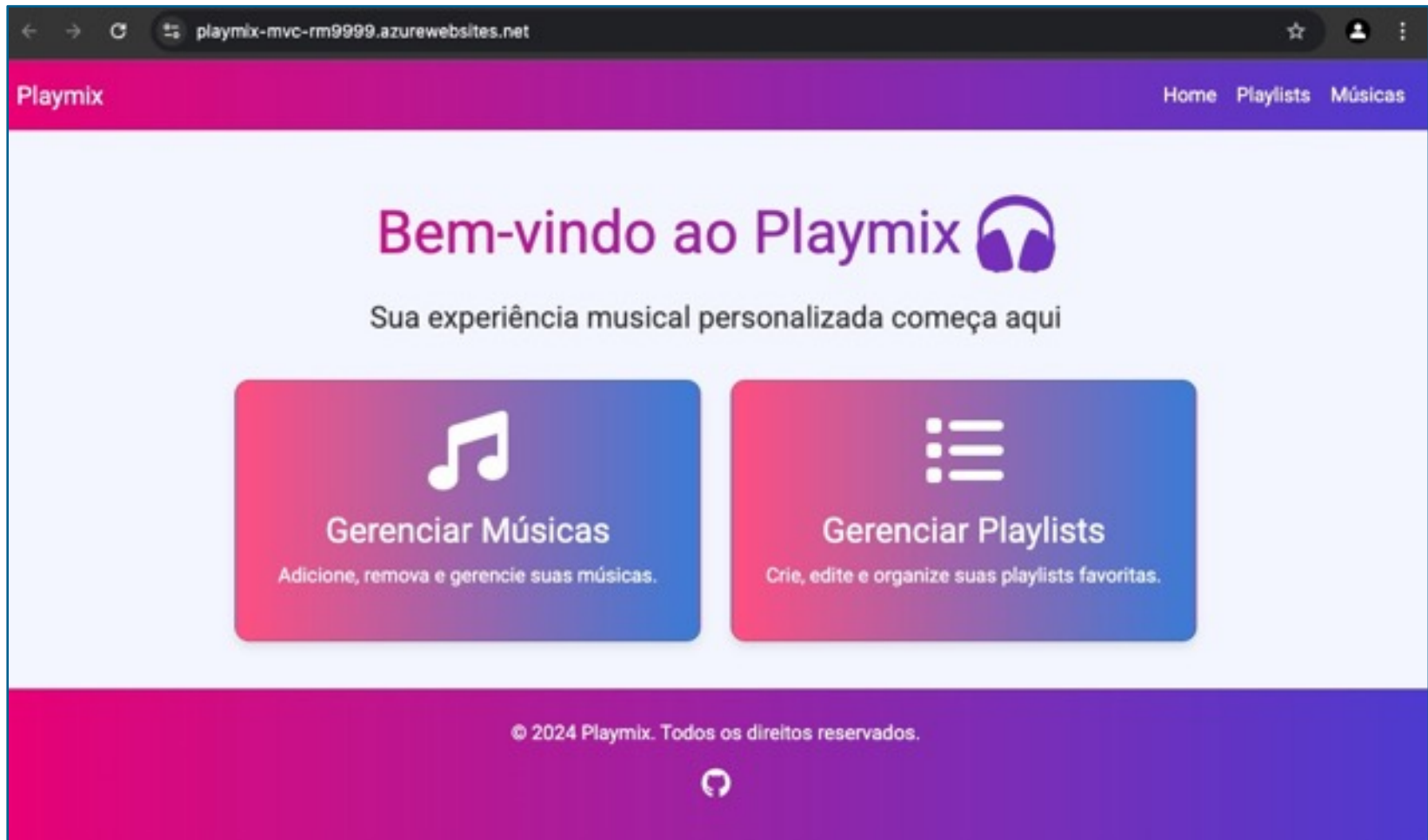
Run details

- Usage**
- Workflow file**

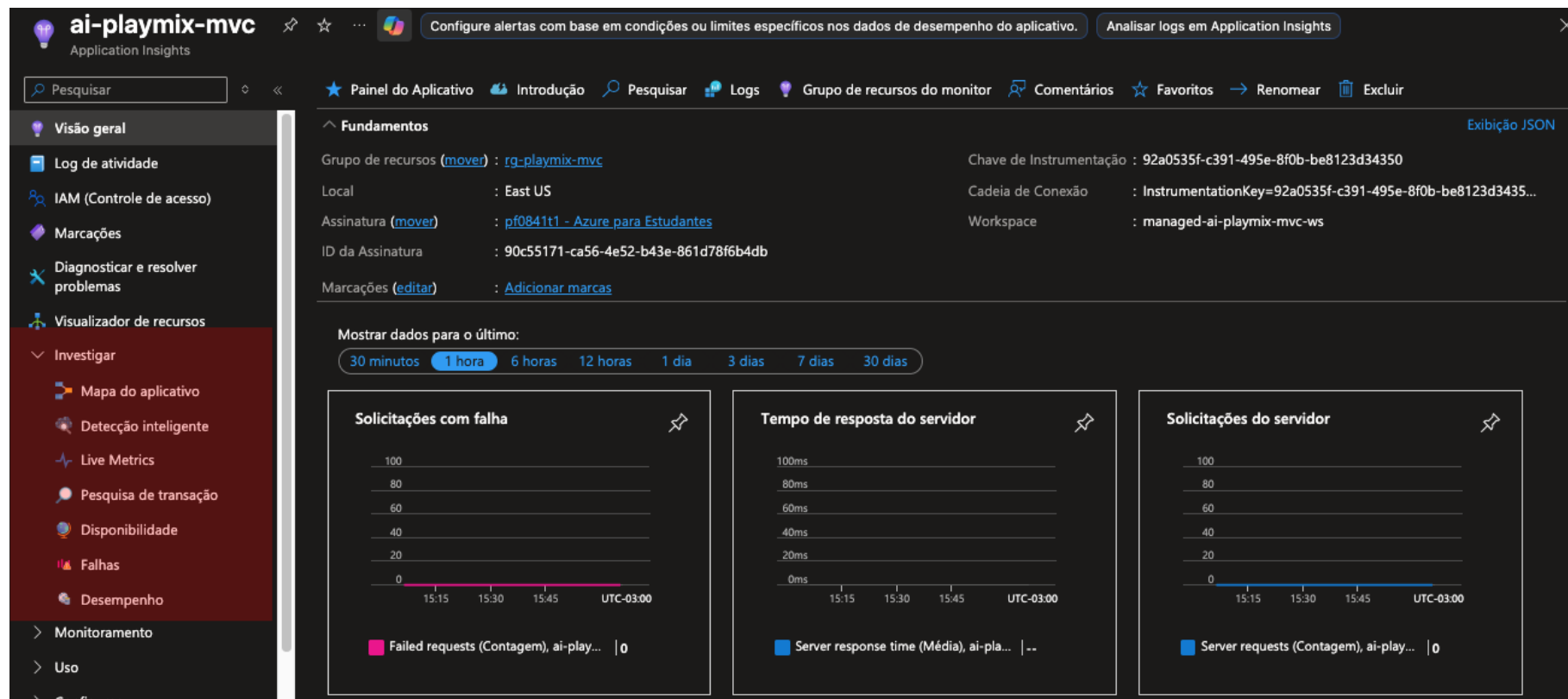
build-and-deploy
succeeded 2 minutes ago in 1m 5s

- > **Set up job**
- > **Run actions/checkout@v2**
- > **Set up Java version**
- > **Build with Maven**
- > **Deploy to Azure Web App**
- > **Post Set up Java version**
- > **Post Run actions/checkout@v2**
- > **Complete job**

Acesse o Link da Aplicação e realize os Testes



Verifique a monitoração no Application Insights



Qual o custo dessa implementação?

Acesse seu **Grupo de Recursos** -> **Gerenciamento de Custos**
-> **Análise de Custo**

The screenshot displays the Microsoft Azure Cost Management interface for the resource group 'rg-playmix-mvc'. The main heading is 'Análise de custo'. The left sidebar contains a navigation menu with the following items: 'Visão geral', 'Log de atividade', 'IAM (Controle de acesso)', 'Marcações', 'Visualizador de recursos', 'Eventos', 'Configurações', 'Gerenciamento de Custos' (highlighted), and 'Análise de custo' (highlighted). The top bar shows the user's email 'pf0841g1@fiap.com.br' and the organization 'FIAP-FACULDADE DE INFORMÁTICA'. The main content area shows the 'EXIBIÇÃO' (Display) dropdown menu set to 'CostByResource', which is highlighted by a red arrow. Other visible elements include the 'Escopo' (Scope) set to 'rg-playmix-mvc', the 'set 2025' date selector, and the 'CUSTO REAL' (Actual Cost) section.

Após os testes delete as tabelas do Oracle

```
DROP TABLE music CASCADE CONSTRAINTS;  
DROP TABLE playlist CASCADE CONSTRAINTS;  
DROP TABLE playlist_music CASCADE CONSTRAINTS;
```

Delete o Grupo de Recursos criado nesse exercício

```
az group delete --name rg-playmix-mvc --yes --no-wait
```

Copyright © 2026 Prof. João Menk

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor)